Application and Components Summary Table			
Percentage of overall contribution: Member 1 name: 55%, Member 2 name: 45% The percentage will be used to scale the grade. Note, if a member does not provide the components in assignment 5, the percentage of contribution will be reduced.			
Provider name	**Page and component type, e.g., aspx, DLL, SVC, etc.**	**Component description: What does the component do? What are inputs/parameters and output/return value?**	**Actual resources and methods used to implement the component and where this component is used.**
Dmytro Ohorodiichuk	ASP.NET Web API (PokerEngine/Controllers/GameController.cs)	REST engine that initializes poker games (PUT /api/games), returns game state (GET /api/games/{id}), and applies player actions with validation (POST /api/games/apply). Inputs include action type and amount; responses are serialized game JSON.	Methods NewGame, GetGame, and ApplyAction build and update Game instances, log events, and progress betting stages. These endpoints feed the Web Forms "Try It" buttons on Default.aspx for starting games, submitting actions, and pulling state for visualization.
Dmytro Ohorodiichuk	WCF Service (PokerBot/PokerBotService.svc)	GetBotDecision accepts a JSON game state (BotRequest.GameStateJson) and returns a BotDecisionResponse containing action type, amount, narration, and raw LLM output.	The service builds a structured Gemini prompt, posts to the Google Generative Language API with an API key header, parses the JSON payload, and falls back to safe defaults on errors. The Web Forms bot "Try It" button in Default.aspx calls this WCF endpoint after downloading the current game state.
Dmytro Ohorodiichuk	User controls (WebApplication1/PlayerDeckView.ascx, WebApplication1/PlayerMoneyView.ascx)	PlayerDeckView renders poker game JSON into an ASCII-style board/hand view and shows error states. PlayerMoneyView binds a player GUID and chip stack for display. Inputs are game JSON (deck view) or parsed player data (money view); outputs are rendered HTML fragments.	RenderFromJson parses board, players, and betting info, formats card glyphs, and toggles error/content panels, while BindPlayer stores identifiers and populates labels. These controls are loaded by the Default.aspx visualization flows that fetch REST game JSON and create controls per player.
Dmytro Ohorodiichuk	DLL (LocalComponents/PasswordHandler.cs)	Library for hashing passwords and verifying hashed input. Inputs are raw or hashed strings; returns hashed string or boolean verification result.	HashPassword uses SHA256 to produce a Base64 hash; VerifyPassword recomputes and compares hashes. The Default.aspx page wires these methods to "Try It" buttons so users can hash or verify sample values via the local DLL.